

The Data Engineering Lifecycle In Depth

Lecture Notes for a 4-Hour Session - Course 1, Week 2

Based on materials by DeepLearning.AI and the textbook
Fundamentals of Data Engineering by Joe Reis & Matt Housley (1)

February 14, 2026

Contents

1	Part 1: The Data Engineering Lifecycle In Depth	3
1.1	Week 2 Overview	3
1.2	Data Generation in Source Systems	3
1.2.1	Common Source System Types	3
1.2.2	The Unpredictability of Source Systems	4
1.3	Ingestion	4
1.3.1	Batch vs. Streaming Ingestion	4
1.3.2	Other Ingestion Considerations	5
1.4	Storage	5
1.4.1	The Raw Ingredients of Storage	5
1.4.2	The Storage Hierarchy	6
2	Part 2: Queries, Transformation, and Serving	7
2.1	Queries, Modeling, and Transformation	7
2.1.1	Queries	7
2.1.2	Data Modeling	7
2.1.3	Transformation	8
2.2	Serving Data	8
2.2.1	Analytics	8
2.2.2	Machine Learning	9
2.2.3	Reverse ETL	9
3	Part 3: The Undercurrents of Data Engineering	11
3.1	Introduction to the Undercurrents	11
3.2	Security	11
3.2.1	The Principle of Least Privilege	11
3.2.2	Data Sensitivity and Minimization	11
3.2.3	Cloud Security Considerations	12
3.2.4	Security as a Culture	12
3.3	Data Management	12
3.3.1	DMBOK Definition and Scope	12
3.3.2	The 11 Data Knowledge Areas	13
3.3.3	Data Governance	13
3.3.4	Data Quality	13
3.4	Data Architecture	13
3.4.1	Definition	14
3.4.2	Principles of Good Data Architecture	14
3.5	DataOps	15
3.5.1	Cultural Practices	15
3.5.2	Three Technical Pillars of DataOps	15
3.6	Orchestration	16
3.6.1	From Manual Execution to Orchestration	17
3.6.2	Directed Acyclic Graphs (DAGs)	17

3.7	Software Engineering	18
3.7.1	The Evolution of Coding in Data Engineering	18
3.7.2	Where You Will Write Code	18
4	Part 4: The Data Engineering Lifecycle on AWS	19
4.1	Lesson Introduction	19
4.2	The Lifecycle Stages on AWS	19
4.2.1	Source Systems	19
4.2.2	Ingestion	19
4.2.3	Storage	20
4.2.4	Transformation	20
4.2.5	Serving	20
4.3	The Undercurrents on AWS	20
4.3.1	Security	20
4.3.2	Data Management	21
4.3.3	DataOps	21
4.3.4	Orchestration	21
4.3.5	Data Architecture	22
4.3.6	Software Engineering	22
5	Week 2 Summary	23

1 Part 1: The Data Engineering Lifecycle In Depth

1.1 Week 2 Overview

This week dives deep into the data engineering lifecycle and undercurrents that were introduced at a high level in Week 1. Recall from Week 1 the core simplification of the data engineer’s mandate: *get raw data from somewhere, turn it into something useful, and then make it available for downstream use cases*. This week, we unpack each of those stages—generation, ingestion, storage, transformation, and serving—in considerably more detail, examining the design decisions, trade-offs, and pitfalls that arise at each step.

The second lesson of the week shifts focus to the six **undercurrents** of the data engineering lifecycle: security, data management, DataOps, data architecture, orchestration, and software engineering. These are not sequential stages but cross-cutting concerns that must be addressed throughout every phase of the lifecycle (1).

While the emphasis remains on building a high-level mental framework—one that will make you more successful in all aspects of your work as a data engineer—the week is not purely theoretical. It concludes with a practical look at how this framework comes together on the AWS Cloud, including a hands-on end-to-end cloud data pipeline lab exercise.

1.2 Data Generation in Source Systems

The first stage of the data engineering lifecycle is the **generation** of data in source systems. As a data engineer, you will be consuming data from various sources. For example, if you are working at an e-commerce company, you might build a data pipeline that consumes sales transaction data from an internal sales database and product catalog data from files. The marketing team might need you to obtain social media data from an API and market research datasets from a data sharing platform. You might even work with data from Internet of Things (IoT) devices, such as GPS trackers for product delivery updates.

Critically, these source systems are typically created and maintained by other teams—software engineers from other internal departments, external vendors, or third-party platforms. As a data engineer, they are largely out of your control (1). However, understanding how these systems work is essential because the data pipelines you build will rely on the data generated from these sources.

1.2.1 Common Source System Types

Databases The most common source systems. These could be **relational databases**, often serving as Online Transaction Processing (OLTP) systems and represented as tables of related data, or other types of **NoSQL systems**, including key-value databases (e.g., Amazon DynamoDB), document stores (e.g., MongoDB), wide-column stores (e.g., Apache Cassandra), and graph databases (e.g., Neo4j) (2). These databases are frequently part of the back end of a web or mobile application.

Files Data in the form of text files, CSV/TSV files, JSON files, audio files (e.g., MP3s), video, or other file types. While individual files might not seem like a “source system,” data engineers frequently need to download or be given access to files—sometimes from third-party vendors—as the starting point of their work.

APIs (Application Programming Interfaces) An API allows you to issue a web request for data and receive it back in a structured format, such as XML or JSON. RESTful APIs and, more recently, GraphQL endpoints are common patterns for programmatic data access (16). For example, a marketing team might need social media engagement data accessible only through a platform’s public API.

Data Sharing Platforms Organizations may set up platforms to share data internally or with third parties. Examples include Snowflake Data Marketplace, AWS Data Exchange, and Databricks Delta Sharing (11). These platforms provide governed access to curated datasets without requiring traditional file transfers or API integrations.

IoT Devices An increasingly common source system. A **swarm of IoT devices** may stream data in real time—for example, GPS trackers gathering location and movement data for product delivery updates, or industrial sensors monitoring equipment health. This streaming data is typically sent to a database or event streaming platform, and the source system owner might make it accessible

via an API or data sharing platform. In other cases, the data engineer may need to ingest and combine all those individual streams directly (1).

1.2.2 The Unpredictability of Source Systems

In a perfect world, the source systems you depend on would deliver the data you need in a consistent and timely manner, such that you could build downstream systems that rely on the predictability of that source system. In the real world, however, source systems are unpredictable. These systems sometimes go down, the team managing the system changes the format or schema of the data, or the schema stays the same but the data itself changes in unexpected ways.

The course instructor shares a telling anecdote: early in his career as a data engineer, he was working with a database maintained by an internal team of software engineers. One day, that team decided to rearrange the columns in their application database without informing him. Columns that his data pipelines relied on were changed, renamed, and some were deleted entirely. This completely halted downstream data workflows and required answering to upset stakeholders.

This illustrates a critical lesson: as a data engineer, you will be more successful if you work directly with source system owners to understand:

- How their systems work and how they generate data
- How that data may be subject to change over time (schema evolution, data drift)
- How those changes will impact the downstream systems you build

Developing good working relationships with the stakeholders of source systems is, in the instructor's words, "an underrated yet crucial part of successful data engineering." This echoes the broader theme from Week 1: technical skills alone are not sufficient; communication and stakeholder management are essential competencies (1).

1.3 Ingestion

Data ingestion means moving raw data from source systems into your data pipeline for further processing. In the instructor's experience, **source systems and data ingestion represent the biggest bottlenecks of the data engineering lifecycle**. If you have followed the advice from the previous section—working directly with source system owners to understand their systems—you will be well positioned to avoid common pitfalls in the ingestion phase.

One of the critical decisions when designing data ingestion processes is **how frequently** you need to ingest the data: how often you need to move data from the source systems into your pipeline for further processing.

1.3.1 Batch vs. Streaming Ingestion

You can think of data being produced as a continuous series of events—clicks on a website, sensor measurements, financial transactions, or something else. Out in the world, events like these are happening continuously. In a sense, you could think of **nearly all data as being streamed at its source**. Batch ingestion is just a convenient way to process this continuous stream of events in large, discrete chunks (2).

Batch Ingestion Processing the stream of events in large chunks, either on a predetermined time interval (e.g., once every hour or once a day) or as data reaches a preset size threshold. For example, you could handle an entire day's worth of data in one single batch. For a long time, batch processing was the default way to ingest data. While there are more options today, batch processing remains a practical and popular approach, particularly in cases where the data is used for analytics and machine learning model training (1).

Streaming Ingestion Ingesting and providing data to downstream systems in a continuous, near real-time fashion—making it available possibly less than one second after it is produced. This requires specific tools such as an **event streaming platform** (e.g., Apache Kafka (9), Amazon Kinesis) or a **message queue** (e.g., Apache Pulsar, Amazon SQS, RabbitMQ) to continuously ingest streams of events. With these tools becoming more ubiquitous, streaming ingestion has become more accessible and popular.

However, streaming ingestion is **not necessarily the best choice for all use cases**, and there are significant trade-offs to consider. Before committing to a stream processing solution, you should ask yourself:

- What actions can you take on real-time data that would be an improvement over batch data?
- Would streaming ingestion cost more than batch in terms of time, money, maintenance, and potential downtime?
- How will choosing stream ingestion over batch (or vice versa) influence the rest of your data pipeline?

The instructor’s advice is direct: “adopt a streaming ingestion system only after you’ve taken the time to identify a business use case that justifies the trade-offs of using it over batch.” Batch ingestion is an excellent approach for many common use cases, like model training and weekly reporting.

The Coexistence of Batch and Streaming. It is important to note that streaming ingestion generally coexists with batch processing. For example, machine learning models are usually trained on a batch basis, but that same training data might have been originally ingested via streaming because of some separate architectural goal, like real-time anomaly detection. Rarely do data engineers have the option to build a *purely* streaming data pipeline with no batch components. Instead, you choose where the boundaries between batch and streaming will occur (1; 2).

1.3.2 Other Ingestion Considerations

Beyond the batch-versus-streaming decision, there are other important nuances to the ingestion stage:

Change Data Capture (CDC) A technique to trigger certain ingestion processes based on data changes in the source system, rather than ingesting on a fixed schedule. CDC detects inserts, updates, and deletes in the source and propagates only those changes downstream. This is far more efficient than full-table scans for large, slowly changing datasets. Common CDC tools include Debezium (open source), AWS DMS, and Oracle GoldenGate (2).

Push vs. Pull Whether a source system **pushes** data to you (e.g., via webhooks or event streams) or you actively **pull** it from the source (e.g., via scheduled API calls or database queries). The choice depends on the capabilities of the source system, latency requirements, and the volume of data involved.

1.4 Storage

Storage is a foundational element that underpins the entire data engineering lifecycle. It is not a single step but a continuous process: data is stored after ingestion, between transformations, and before being served. In your work as a data engineer, the function, performance, and limitations of the systems you build will have a lot to do with the storage solutions you choose to support those systems.

To ground this in everyday experience: think about all the different ways you interact with data storage systems on a daily basis. On your laptop, you create or delete files, moving data around on your hard disk or SSD. When you open applications, you load them into RAM for faster access. You download files from the Internet or have files backed up to cloud storage. With almost any action you take on a digital device, you are interacting with data storage systems—and you might encounter their limits, like running out of space on your phone or attempting to send files that are too large.

1.4.1 The Raw Ingredients of Storage

Magnetic Disk (HDD) Still forms the backbone of modern storage systems, primarily because of **low cost**. At the time of the course recording, disk storage is about 2–3 times cheaper than solid-state storage. While you may not have read data from a floppy disk recently, magnetic disks remain central to data center infrastructure (1).

Solid-State Drives (SSDs) Flash memory offering faster read/write speeds than magnetic disks. SSDs are commonly found in consumer laptops and smartphones and are increasingly used in enterprise storage for workloads requiring higher I/O performance.

RAM (Random Access Memory) Offers much faster read and write speeds than SSDs or disks, making it a critical component of many applications and architectures—particularly for caching, in-memory databases (e.g., Redis, Apache Ignite), and real-time analytics engines. However, RAM storage could be **30–50 times more expensive** than solid-state storage. It is also **volatile**: if your system loses power, memory is typically lost instantly (depending on the type of RAM used) (1).

Other Raw Ingredients In modern cloud architectures, data is commonly distributed across multiple clusters and data centers. This means that **networking**, **CPU**, **serialization** (converting data structures to byte streams), **compression** (reducing data size), and **caching** (storing frequently accessed data in faster storage) are all critical raw ingredients for storing data in modern distributed systems (2).

In most modern architectures, data will pass through magnetic storage, solid-state drives, and memory as it works its way through the various processing phases of a data pipeline.

1.4.2 The Storage Hierarchy

If you think about storage as a hierarchy, it can be arranged in three tiers:

1. **Raw Ingredients (Bottom):** The physical components (disk, RAM, SSD) and non-physical components (networking, serialization, compression, caching) that exist inside servers and clusters.
2. **Storage Systems (Middle):** Built on top of raw ingredients, allowing systems to ingest and retrieve data using different access protocols. These include:
 - **Database management systems** (relational, NoSQL, time-series, etc.)
 - **Object storage platforms** (e.g., Amazon S3)
 - **Table formats** like Apache Iceberg (12) and Apache Hudi for managing large analytical datasets
 - **Cache and memory-based storage systems** (e.g., Redis, Memcached)
 - **Streaming storage** (e.g., Kafka topics as durable storage)
3. **Storage Abstractions (Top):** Combinations of storage systems that allow you to achieve high-level data storage needs without worrying about as many low-level details. The three major abstractions are:
 - **Data Warehouse:** Optimized for structured, analytical queries (e.g., Amazon Redshift, Google BigQuery, Snowflake). Originally conceptualized by Bill Inmon in the 1990s to separate analytical workloads from operational databases (4).
 - **Data Lake:** A centralized repository for storing raw data at any scale in its native format (structured, semi-structured, or unstructured), typically on object storage like Amazon S3 (1).
 - **Data Lakehouse:** A more recent architectural pattern that combines the flexibility of a data lake with the performance and governance features of a data warehouse, using open table formats like Apache Iceberg or Delta Lake (11).

As a data engineer, it is entirely possible that you will spend much of your time operating at or near the **top** of this hierarchy, meaning you will not be required to think about the details of exactly how your data is moving between different storage components and systems. However, you will be most effective in your work if you take the time to understand the inner workings, capabilities, and limitations of your entire storage solution, right down to the raw ingredients.

A Cautionary Tale. The instructor shares an experience consulting for a team that failed to consider storage details. They needed to move a large dataset into their data warehouse and unwittingly chose to use direct row inserts for each individual row of data—ingesting and writing one row at a time. This turned out to be not only very slow but also very expensive, since direct row inserts are generally charged on a per-usage basis. They discovered their error and moved to a bulk ingestion approach, but only after they had wasted significant time and **burned through half their annual data warehousing budget in the span of a week**. The moral: it will be to your advantage to be “storage savvy” as a data engineer (1).

2 Part 2: Queries, Transformation, and Serving

2.1 Queries, Modeling, and Transformation

The transformation stage of the data engineering lifecycle is where you, as a data engineer, **start to add value**. The stages that come before transformation—ingesting and storing raw data from source systems—do not directly add value for downstream stakeholders. Recall the big picture: you get raw data from somewhere, *turn it into something useful*, and then make it available to end users. Transformation is the “turn it into something useful” stage.

In terms of what “useful” looks like, consider two examples:

- A **business analyst** tasked with reporting on daily sales across a range of products. They might need customer IDs, product names, prices, quantities, and times of sale. While business analysts are often fluent in SQL, they rely on data engineers to transform the raw data and provide it in a format that is quick and easy to query.
- A **data scientist or ML engineer**. In addition to SQL, they might be fluent in many approaches to data transformation, but their core function is using data for predictive analytics. You can provide tremendous value by handling the transformation of data into structures and features that can be used directly for model training or analysis.

While this stage is called “transformation,” it is actually composed of three interrelated parts: **queries, modeling, and transformation**. Queries and modeling are included as distinct from transformation because they are critical components of any data pipeline that add significant value when done well and present serious risks when done poorly (1).

2.1.1 Queries

When you query data, you issue a request to read records from a database or other storage system. For example, you may need to query tabular and semi-structured data sitting in a cloud data warehouse. While there are many query languages, these courses focus on **Structured Query Language (SQL)**, which remains the most popular and universal query language for data work (6).

A query may involve cleaning, joining, and aggregating data across many datasets. You may use SQL expressions to filter data so that only specific records are retrieved. Importantly, there is **more than one way to write a query**, and poorly written queries can have serious negative consequences:

- **Impacting source database performance:** An expensive query against a production OLTP database can slow down the application that depends on it, affecting end users.
- **Row explosion:** A query that includes a JOIN between tables can produce many more records than anticipated (e.g., a Cartesian product from an incorrect or missing join condition), which can overwhelm your storage infrastructure.
- **Downstream delays:** Slow or overly broad queries can cause cascading delays in reporting or analytics pipelines.

In practice, the instructor notes that “most data engineers can read and write SQL but are unfamiliar with how queries work under the hood.” Understanding query execution plans, indexing strategies, and the internal mechanics of query optimizers can have a significant impact on the performance of the architectures you build (1; 6).

2.1.2 Data Modeling

A **data model** represents the way data relates to the real world. Data modeling involves deliberately choosing a coherent structure for your data and is a crucial step in making data useful for the business (3).

Normalization Data from upstream relational source databases is often **normalized**—organized into separate tables for distinct entities (e.g., product information, order details, customer information) with relationships defined through foreign keys. Normalization reduces data redundancy and helps maintain data integrity, following Edgar F. Codd’s relational model and the normal forms (1NF, 2NF, 3NF, BCNF) (5; 6). This structure is optimal for transactional workloads (OLTP) but can be complex and slow for analytical queries that require joining many tables.

Denormalization To serve an analyst’s reporting needs, you might need to **denormalize** the data—combining tables and introducing controlled redundancy to model the data in a way that allows quick and efficient querying. Ralph Kimball’s **dimensional modeling** approach, using **star schemas** and **snowflake schemas** with fact and dimension tables, is the classic technique for building analytical data models in data warehouses (3).

A good data model is designed to best reflect your organization’s processes, definitions, workflows, and logic. For example, the term “customer” might mean different things to different departments in your company—a paying subscriber to the product team, a lead to the sales team, or an account holder to the finance team. To be successful with data modeling, you need to work with stakeholders to understand their terminology and business goals. This alignment between data models and business semantics is a key aspect of **data governance** (see Section 3.3).

2.1.3 Transformation

Beyond querying and modeling, data must also be **transformed**—manipulated, enhanced, and saved for downstream use. Data is typically transformed multiple times throughout the data engineering lifecycle, at various stages and for various purposes (1):

1. **At the source (pre-ingestion):** A timestamp may be added to a record while it is still on a source system, or an application may enrich a log entry with contextual metadata before writing it.
2. **In-flight during ingestion:** Transformations applied while data is being moved from source to storage—for example, filtering out irrelevant records, converting data types, or applying basic validation rules during a streaming ingestion pipeline.
3. **Immediately after ingestion (staging):** Basic transformations to map data to correct types and put records into standardized formats. This often occurs in a “raw” or “staging” zone of a data lake.
4. **Within a streaming pipeline:** Enriching a record with additional fields and calculations (e.g., joining with reference data, computing derived metrics) before it is sent to a data warehouse or served in real time.
5. **Further downstream (analytical transformation):** Schema transformation, denormalization, large-scale aggregation for reporting, or featurization for machine learning model training. Tools like **dbt** (data build tool) (13) have popularized the ELT pattern, where raw data is loaded first and then transformed within the warehouse using SQL.

This multi-stage transformation process reflects the reality that different consumers need data in different forms, and the “right” transformation depends entirely on the downstream use case.

2.2 Serving Data

Once you have ingested, transformed, and stored your data, you are ready for the final stage of the data engineering lifecycle: **serving data**. This stage is about more than just making data available—it is when you give stakeholders the opportunity to extract business value from data. As emphasized throughout this course, value means different things to different stakeholders (1).

Generally, data has value when it is used for practical end use cases. The three primary categories of serving are analytics, machine learning, and reverse ETL.

2.2.1 Analytics

Analytics is the process of identifying key insights and patterns within data. As a data engineer, you are almost guaranteed to serve data that powers one or more of the three most common forms of analytics:

Business Intelligence (BI) The most traditional form of analytics. Analysts explore **historical and current business data** to discover insights. Data is ultimately presented in the form of reports or dashboards that help users make strategic and actionable decisions (3). For example:

- Sales and marketing teams use BI reports and dashboards to spot patterns and trends, monitoring metrics like marketing campaign engagement, regional sales, and customer experience.
- An analyst might observe a sudden spike in product returns. They could investigate existing reports or dashboards to compare this phenomenon with historical trends, pull more data by running SQL queries against the database you provided, or ask you to serve up additional data for **ad hoc analytics**.

Operational Analytics In contrast to the reflective, insight-driven nature of BI, operational analytics is about **monitoring real-time data for immediate action**. For example, an e-commerce platform team might need to monitor a dashboard with real-time performance metrics for their website. If the website goes down—whether because of a spike in user traffic or a data center going offline—they need to know immediately so they can triage the situation and restore service. The data engineer’s role is to ingest, transform, and serve event data from the website’s application logs to populate the platform team’s dashboard.

Embedded Analytics External or **customer-facing analytics**—a somewhat newer trend. You have most likely interacted with embedded analytics applications yourself:

- Your bank might provide dashboards showing historical trends in your spending or how your purchases break down across categories like food, retail, and utilities.
- A smart thermostat connected to a mobile application might show the current temperature inside your home as well as historical metrics like temperature over time.

As a data engineer working on embedded analytics, your job is serving real-time and historical data for use in user-facing applications. This often requires higher standards for latency, availability, and data freshness than internal analytics (1).

While BI and operational analytics are internally focused data practices that have been in use for many decades, embedded analytics represents a growing area where data engineering intersects with product engineering.

2.2.2 Machine Learning

With the rise of machine learning in recent decades, it is quite likely that your role as a data engineer will involve serving data for ML use cases. The course treats machine learning as separate from other serving use cases because it can involve additional complexities (1):

- Serving **feature stores**—centralized repositories of curated, pre-computed features—that facilitate model training and ensure consistency between training and serving environments (25).
- Serving data for **real-time inference**, where models need to score new data points with minimal latency.
- Supporting **metadata and cataloging systems** that track data history, lineage, and provenance—critical for model reproducibility and regulatory compliance.
- Managing the **training-serving skew** problem, where discrepancies between training data and production data lead to degraded model performance.

2.2.3 Reverse ETL

With reverse ETL, you take transformed data, analytics outputs, and perhaps machine learning model output and **feed it back into source systems**. This creates a feedback loop that enriches operational systems with analytical insights. A concrete example:

1. Ingest data from a **Customer Relationship Management (CRM)** system—this might include names, contact information, form data, and other relevant client information.
2. Transform the data into the appropriate format and store it in a data warehouse.
3. Analysts retrieve the data to train a **lead scoring model**, which attempts to identify which clients are the most promising for various engagements or product offerings.

4. The model results are returned to the data warehouse and then **pushed back into the CRM** as an enhancement of the client data already stored there—for example, adding a “lead score” field that sales representatives can use to prioritize their outreach.

As the instructor notes, “the name reverse ETL for this process is not so much an attempt to describe what’s going on as it is just a lack of a better name to describe this pretty longstanding process.” Tools like Census, Hightouch, and Polytomic have emerged specifically to facilitate reverse ETL workflows (1). Regardless of what it is called in the future, this practice is increasingly common and likely to be part of your role.

3 Part 3: The Undercurrents of Data Engineering

3.1 Introduction to the Undercurrents

As the field of data engineering has matured, the continued abstraction and simplification of tools has expanded the scope of the role. Whereas just a decade ago the role of a data engineer was primarily focused on the technology layer, modern data engineering encompasses far more than just tools and technologies. In other words, **the field is moving up the value chain** (1).

Modern data engineering incorporates traditional enterprise practices such as data management and cost optimization, as well as newer practices like DataOps. The *Fundamentals of Data Engineering* textbook describes these as the **undercurrents** of the data engineering lifecycle—a set of practices that apply across the entire lifecycle, not as sequential stages but as cross-cutting concerns:

1. Security
2. Data Management
3. DataOps
4. Data Architecture
5. Orchestration
6. Software Engineering

3.2 Security

Before getting into how security applies to your role as a data engineer, consider how security concerns apply to your own personal data every day. You probably do not give out your bank account information to just anyone, do not publish the passwords to all your online accounts, and would not give the keys to your house to someone you do not know or trust.

In your role as a data engineer, you are entrusted with sensitive data—whether that is the personal and private information of your clients or proprietary business information. The owners of that data are trusting that you will keep their information safe. Security is about bringing together the right set of **principles, protocols, and cultural practices** to ensure the security of the systems you build (1).

3.2.1 The Principle of Least Privilege

When managing a data pipeline and serving data to end users, you will need to give access to data and system resources to other users or applications. The key security principle to keep in mind is the **Principle of Least Privilege** (17):

Give users or applications access to only the essential data and resources they need to do their jobs, and only for the duration that is required.

Critically, this principle applies not only to others in your organization but **also to yourself**. Just as you do not give everyone team administrator access, when it comes to your own work:

- Do not operate from the root shell when it is not necessary.
- Do not use administrator or super-user permissions unless it is absolutely essential.
- Use role-based access control (RBAC) to scope permissions to specific tasks and time windows.

3.2.2 Data Sensitivity and Minimization

Beyond access control, you need to think about **data sensitivity**. Adhering to the Principle of Least Privilege means making sensitive information visible to users only when it is absolutely necessary. Beyond that, the best way to protect sensitive data is **not to ingest it into your system in the first place**. If you do not have a clear purpose for ingesting and storing sensitive data, simply do not do it—you will have entirely removed the risk of accidentally leaking that data. This principle of **data minimization** is also a core tenet of privacy regulations such as the EU’s General Data Protection Regulation (GDPR) (18) and the California Consumer Privacy Act (CCPA).

3.2.3 Cloud Security Considerations

In today’s cloud-centric world, security takes on additional dimensions that require understanding:

- **Identity and Access Management (IAM):** Managing who can access what resources, using services like AWS IAM with roles, policies, and temporary credentials (see Section 4.3.1).
- **Encryption:** Protecting data both at rest (stored data) and in transit (data moving between systems) using encryption standards such as AES-256 and TLS.
- **Networking protocols:** Configuring Virtual Private Clouds (VPCs), security groups, and network ACLs to control network-level access to resources.

3.2.4 Security as a Culture

Security is not just about principles and protocols—it is also about **people**. Security starts and ends with you, the individual, as well as other individuals across your organization. The instructor recommends adopting a **defensive mindset**: always be cautious when asked for credentials or for sensitive and confidential data, imagine potential attack and leak scenarios, and design your data pipelines and storage systems with them in mind.

When it comes to real-world data leaks, individual people have been the source of many of the biggest security breaches:

- People ignoring basic precautions, like sharing passwords insecurely with others
- People falling victim to **phishing attacks**, where an attacker tries to steal sensitive information by impersonating an authority figure or someone of trust
- People acting irresponsibly while working in company systems and data
- Data engineers accidentally leaving an **AWS S3 bucket**, a server, or a database exposed to the public Internet when that was not the intention

The instructor observes that “all too often, organizations are set up with a facade of security—maybe a set of checklists to show they’re compliant with regulations or compliance standards, but a deeper spirit of security is lacking in the organization’s culture.” This approach of adhering to the *letter* of security without a genuine culture and spirit of security is what the textbook calls **security theater** (1). True security emerges from a culture where every team member recognizes their role in protecting data and everyone, from top to bottom, embraces security as a priority and a habit.

3.3 Data Management

Data can be an incredibly valuable business asset, but only when it is managed properly. Data management is so important that there is an international organization, the **Data Management Association International (DAMA)**, dedicated entirely to providing resources for companies and individuals to get data management right. DAMA publishes the **Data Management Body of Knowledge (DMBOK)** (7), a comprehensive reference that covers the full scope of data management disciplines.

3.3.1 DMBOK Definition and Scope

The DMBOK defines data management as:

The development, execution, and supervision of plans, programs, and practices that deliver, control, protect, and enhance the value of data and information assets throughout their lifecycles.

As a data engineer, you do not need to memorize everything in the DMBOK—it is a *reference volume*. You will focus on a subset of data management tasks and share the full responsibility of data management with other teams, including software engineering, IT, and others.

3.3.2 The 11 Data Knowledge Areas

The DMBOK breaks down data management into 11 knowledge areas (7):

1. Data Governance
2. Data Architecture
3. Data Modeling and Design
4. Data Storage and Operations
5. Data Security
6. Data Integration and Interoperability
7. Document and Content Management
8. Reference and Master Data
9. Data Warehousing and Business Intelligence
10. Metadata Management
11. Data Quality

Notably, **data governance touches all other areas** of data management. Many of the other knowledge areas interact with each other through data governance practices.

3.3.3 Data Governance

According to *Data Governance: The Definitive Guide* (8):

Data governance is first and foremost a data management function to ensure the quality, integrity, security, and usability of the data collected by an organization.

Data governance covers a lot of ground—everything from data security and privacy to data quality and usability.

3.3.4 Data Quality

A key concern within governance is **data quality**, which is closely related to concepts like integrity, usability, and reliability. While data quality is a deep and nuanced topic, at its core the concept is relatively straightforward:

- **High-quality data** is accurate, complete, discoverable, and available in a timely manner. Beyond that, high-quality data represents exactly what stakeholders expect it to represent in terms of well-defined schema and data definitions. Data that conforms to standards of quality is a powerful tool in decision-making and adds great value to an organization.
- **Low-quality data** might be inaccurate, incomplete, or otherwise unusable. Low-quality data can cause stakeholders to waste their time, make poor decisions, or—as the instructor bluntly notes—“even fire their entire data team.”

Modern data quality frameworks emphasize dimensions such as accuracy, completeness, consistency, timeliness, uniqueness, and validity (7). Tools for data quality monitoring include Great Expectations, Soda, Monte Carlo, and dbt tests.

3.4 Data Architecture

You can think of data architecture as a **roadmap or blueprint** for your data systems. In Week 1, we discussed requirements gathering and how to translate stakeholder needs into specific requirements for making design and technology choices. In order to map requirements to a successful design, you need to **think like an architect**.

In some organizations, a dedicated **data architect** establishes the design and passes it on for data engineers to implement. In other circumstances—such as a small startup—you may be both the architect and the engineer. In any case, being able to think like an architect will make you more successful in your role (1).

3.4.1 Definition

In *Fundamentals of Data Engineering*, Reis and Housley define data architecture as:

The design of systems to support the evolving data needs of an enterprise, achieved by flexible and reversible decisions reached through a careful evaluation of trade-offs.

Let us unpack this definition:

Evolving needs: A good design supports not only the data needs of today but also tomorrow. In practice, this means architecture is an **ongoing effort**, rather than something you do once and are done with.

Flexible and reversible decisions: The data needs of your enterprise may evolve in ways you had not anticipated, and you will need to update your architecture over time. If your initial design choices were flexible and reversible, you will have a much easier time evolving your architecture.

Careful evaluation of trade-offs: These might include trade-offs in performance, cost, scalability, complexity, or other parameters.

The Cloud Advantage. Back when essentially all data architectures were built as on-premises systems, making flexible and reversible decisions was much harder—in some cases, impossible. If you decided to purchase and install millions of dollars’ worth of server hardware, you were committed to that system for years. Nowadays, with most data architectures being built in the cloud, you can change your mind about technology choices provided you have made flexible and reversible decisions to begin with (1).

3.4.2 Principles of Good Data Architecture

The textbook identifies nine principles that will be revisited throughout the specialization:

1. **Choose common components wisely.** Common components are the parts of your architecture used by multiple individuals and teams across your organization. A good choice provides the right set of features for individual projects and simultaneously facilitates collaboration between teams. For example, choosing a data warehouse that serves both the analytics and data science teams avoids unnecessary data duplication and inconsistency.
2. **Plan for failure.** A good architecture is designed not only for the case where everything is working as expected, but also for when things break. This includes designing for redundancy, implementing circuit breakers, and planning disaster recovery procedures. The *AWS Well-Architected Framework* (32) formalizes this as the **Reliability** pillar.
3. **Architect for scalability.** Scalable systems can scale up to meet demand as needed and scale down to minimize costs when demand recedes. When you build scalability into your architecture, you can be responsive to changing demand and optimize for cost simultaneously. Cloud services with auto-scaling capabilities (e.g., Amazon Redshift’s concurrency scaling, serverless compute) embody this principle.
4. **Architecture is leadership.** While this principle may not directly apply to you in your current role, thinking like an architect and seeking mentorship from data architects will help you lead and mentor other team members as your skills develop. Eventually, you may occupy the data architect role yourself.
5. **Always be architecting.** Architecture design is not something that happens only once. Instead, you will be constantly evaluating your systems against the evolving needs of your organization and re-architecting on an ongoing basis.
6. **Build loosely coupled systems.** A loosely coupled system is built from individual components that can be easily swapped out for others without having to re-architect the whole system. This aligns with the microservices philosophy in software engineering and enables teams to make independent technology choices (28).
7. **Make reversible decisions.** Choosing to build with easily interchangeable components means you are making a set of reversible decisions. If the needs of your organization evolve, you can swap out components of your architecture to meet new design specifications without being locked in.

8. **Prioritize security.** Security principles like Least Privilege (see Section 3.2) and the **Zero Trust** model—where no user, device, or network is trusted by default, and every access request must be verified—are central to your role (19).
9. **Embrace FinOps.** The cost structure of data has evolved dramatically in the cloud era. **FinOps** is a movement to bring together the business priorities of finance and DevOps (or DataOps). On the cloud, most data systems are pay-as-you-go and readily scalable. By embracing FinOps, you can design your systems to be simultaneously optimized for cost and potential revenue generation (29). This connects to the Week 1 discussion of Total Cost of Ownership (TCO) and Total Opportunity Cost of Ownership (TOCO).

3.5 DataOps

Around 2007, a framework called **DevOps** emerged in software development to break the silos between software development teams (who write and test code) and software deployment teams (who deploy and maintain code) (20). DevOps borrows from other well-known methodologies, including **lean manufacturing** (22) and **agile software development** (21), to accomplish:

- Removal of bottlenecks
- Reduction of waste
- Quick identification of problems
- Rapid iteration

The DevOps movement has resulted in increased release cycles and enhanced quality for software products. As the data field has matured, a similar approach—known as **DataOps**—has been adopted for the development of data products. Similar to how DevOps improves the development process and quality of software products, DataOps aims to improve the development process and quality of **data products** (any data or data system that you provide to end users) (1; 24).

3.5.1 Cultural Practices

DataOps is first and foremost a set of **cultural habits and practices**, borrowed directly from the agile methodology—a project management framework focused on delivering work in incremental and iterative steps (21):

- Prioritizing communication and collaboration with business stakeholders
- Continuously learning from successes and failures
- Taking an approach of rapid iteration to work toward improvements to systems and processes

3.5.2 Three Technical Pillars of DataOps

The technical elements of DataOps rest on three key pillars:

Pillar 1: Automation One of the DevOps practices that accelerated the software build lifecycle is **Continuous Integration and Continuous Delivery (CI/CD)** (23). With CI/CD, developers automate many of the manual processes required to build, test, and deploy code, resulting in faster review and deployment cycles and fewer errors.

DataOps employs a similar automation framework to data processing. The high-level goal of automated change management remains the same for code, configuration, and environment. In addition, DataOps is focused on change management for **data processing pipelines and the data itself**.

To illustrate, imagine you are the first data engineer at a small organization, tasked with building a data pipeline that ingests from multiple source systems (a database, files, an API, a data sharing platform), performs in-flight transformations, stores the data, and serves two end use cases (analytics and machine learning). The progression of automation typically follows three stages:

1. **Manual execution:** Manually starting each ingestion process, then once complete, manually executing each subsequent step. Reasonable for quick prototyping but prone to errors and inefficient in the long run.
2. **Pure scheduling:** Setting each task to start at a particular time (e.g., all ingestion tasks at midnight, transformation tasks two hours later). Better than manual execution, but fragile: if ingestion fails or takes longer than expected, downstream tasks may process incomplete or stale data.
3. **Orchestration:** Adopting a framework like Apache Airflow that checks dependencies between tasks within your pipeline before each task is run, automatically starts subsequent tasks once previous ones complete, notifies you of errors, and prevents dependent tasks from starting when they should not (see Section 3.6).

Pillar 2: Observability and Monitoring Any data pipeline you set up is bound to fail eventually. To quote Werner Vogels, the CTO of Amazon Web Services: *“Everything fails all the time.”* This means that if you are not closely observing and monitoring your data systems, you will be caught unaware when they fail.

In a worst-case scenario, you might only become aware of system failures when downstream stakeholders discover problems on their own—for example, in their reports or analytics dashboards. The instructor notes that in his consulting work, he has “seen countless cases of bad data lingering in reports for **months or even years** due to undiscovered failures in data processing systems.” Such failures waste time and money, lead to ill-informed decisions, and can ultimately cost you your job if stakeholders lose trust in your work.

Key observability practices include:

- **Data quality checks:** Automated validation of data completeness, freshness, schema conformity, and statistical distributions (25).
- **Pipeline health metrics:** Monitoring task durations, success/failure rates, data volumes, and resource utilization.
- **Alerting:** Proactive notifications (via email, Slack, PagerDuty, etc.) when anomalies or failures are detected.
- **Data lineage:** Tracking where data came from, how it was transformed, and where it is consumed, enabling root-cause analysis when issues arise.

Pillar 3: Incident Response Using the observability and monitoring capabilities you have set up to rapidly identify the root causes of an incident and resolve it as quickly and reliably as possible. As the instructor emphasizes, “things will break, and it’s only a matter of time before they break.”

Incident response is not just about the technology and tools you use to identify and respond to an issue. It is also about **open and blameless communication** and coordination of the efforts of members of the data team responding to the incident, as well as others across the organization. Practices from Site Reliability Engineering (SRE), such as blameless postmortems and error budgets, are increasingly relevant to data engineering teams (26).

A data engineer should be **proactively finding issues before they are reported** by other stakeholders. DataOps is a relatively new set of ideas that is still a work in progress, and not all organizations have adopted DataOps best practices (1).

3.6 Orchestration

When you think of the word “orchestration,” you might picture a conductor guiding an orchestra—signaling when various instruments should be featured, shifting tempo and intensity, all in an effort to make good music. Like an orchestra, a data pipeline has a lot of moving parts that must be coordinated to get a good result. As a data engineer, **you are the conductor** in charge of coordinating and managing the tasks in your data pipelines (1).

Orchestration is both a central component of DataOps (see Section 3.5) and a separate undercurrent spanning the entire data engineering lifecycle. It is considered a separate undercurrent because of how critical it is to modern data architectures and pipelines.

3.6.1 From Manual Execution to Orchestration

Manual Execution Manually triggering each step in your data pipeline when you need it to run. This might help in prototyping various aspects of your system, but in the long run, it is not a sustainable method for data processing.

Pure Scheduling Setting tasks to run automatically at particular times or frequencies. For example, scheduling ingestion tasks to start at the same time each day, then scheduling downstream transformation steps to kick off an hour after you expect the ingested data to be present. Pure scheduling has been widely used historically, but it has significant limitations:

- If a scheduled ingestion task fails, downstream transformation tasks may also fail.
- If a transformation task takes longer than expected and the next task kicks off before it finishes, you can end up with incomplete, stale, or otherwise problematic data propagating through your pipeline.

Orchestration Frameworks Not that long ago, orchestration frameworks more sophisticated than pure schedulers were only available to large enterprises with the resources to build custom solutions. Today, several open-source frameworks make sophisticated orchestration accessible to teams of any size:

- **Apache Airflow** (14): Currently the most popular framework, created at Airbnb in 2014 and later donated to the Apache Software Foundation.
- **Dagster** (15): Focuses on software-defined assets and type-safe data dependencies.
- **Prefect**: Emphasizes developer experience with a hybrid execution model.
- **Mage**: Aims for simplicity with a notebook-style interface for pipeline development.

These frameworks allow you to:

- Build in complex **dependency-based task triggering** (verify that previous tasks completed before starting the next).
- Set up **event-driven triggers** (e.g., start an ingestion step when a certain amount of new data is available in the source database).
- Configure **monitoring and alerting** (notify you if a task fails or has not completed by a certain time).
- Enable **automatic verification and deployment** of new pipeline components, similar to CI/CD for software (23).

3.6.2 Directed Acyclic Graphs (DAGs)

Many orchestration frameworks require you to set up your data pipeline as a **Directed Acyclic Graph (DAG)**—a concept from graph theory that, despite its intimidating name, is really just a way to describe how data flows through your pipeline:

Directed The flow of data goes only in one direction—from source to destination, from upstream tasks to downstream tasks.

Acyclic There are no loops. Data does not flow back to a previous step, preventing infinite cycles.

Graph The pipeline is composed of **nodes** (tasks) and **edges** (data flow between tasks).

You can think of a DAG as a **flow chart** for how data moves through your pipelines. For example, a typical DAG might show four source systems feeding into extraction tasks, with data flowing to a storage step, then branching into two paths—one for ML transformations and one for analytics transformations—each ending in its own serving layer.

Within your orchestration framework, you construct and deploy DAGs, specifying criteria and dependencies for how each task should be triggered and what monitoring and alerts you want to set up. The DAG abstraction makes pipeline dependencies explicit, auditable, and manageable (1).

3.7 Software Engineering

Of all the undercurrents, perhaps the most straightforward one to understand is software engineering. The core message is simple: **as a data engineer, you need to know how to read and write code**. Not just hacking together code that does what you need right now, but writing **production-grade code** that is clean, readable, testable, and deployable (27; 1).

3.7.1 The Evolution of Coding in Data Engineering

In the not-too-distant past, there was no such thing as “data engineering” as an official profession. There were just software engineers who occasionally dealt with data in their own work. Eventually, as businesses recognized the value of data, software engineers began taking on various aspects of data engineering. With the growing diversity and volume of data in recent decades, the data-oriented component of software engineering became much more intensive and eventually emerged as its own field.

Over the years, those software engineers doing data engineering built a variety of fantastic solutions, such that nowadays you have access to a wide range of managed services and applications that allow you to more efficiently get the job done. This is good news: it allows you to spend more time focused on the most important aspects of adding real value to your organization. In a sense, existing services allow you to **move up the value chain**.

This also means that as a data engineer today, **you are often required to write much less code** than your software-engineering-oriented predecessors of a decade or two ago. However, this does not mean coding is less important—in fact, it is more important than ever that the code you write is of top quality.

3.7.2 Where You Will Write Code

- **Core data processing code** at all stages of the lifecycle—from ingestion to transformation and serving. You will need to be proficient in frameworks and languages such as:
 - **SQL**: The lingua franca of data (6).
 - **Apache Spark**: For distributed data processing at scale (10).
 - **Apache Kafka**: For stream processing (9).
- **Programming languages**: You are likely to encounter Python (the most common general-purpose language in data engineering), Java Virtual Machine (JVM) languages like Java or Scala, and Bash for command-line operations. You may also encounter newer languages like Rust or Go. If you focus on building strong foundational software engineering skills, you will not have much trouble moving between languages.
- **Open-source framework development**: You may adopt an open-source framework to solve a particular problem and end up further developing it for your specific use case. As long as you write good code, you can make a pull request and add your contributions to the project to help others solve similar problems.
- **Infrastructure as Code (IaC)**: Tools like Terraform (30), AWS CloudFormation, and Pulumi allow you to define and provision infrastructure using code, making your deployments reproducible and version-controlled.
- **Pipeline as Code**: Defining your data pipelines programmatically (e.g., Airflow DAGs in Python), enabling version control, code review, and automated testing of pipeline logic.
- **General-purpose problem solving**: At all stages of the lifecycle, you will write code for everyday tasks—data validation scripts, one-off data fixes, monitoring utilities, and more.

The instructor’s advice is practical: “It’s well worth your time to make friends with the software engineers in your organization and learn from them how to write great code.” Building strong foundational software engineering skills—including version control (Git), testing (unit tests, integration tests), code review practices, and clean code principles (27)—ensures you can contribute effectively regardless of the specific tools or languages required.

4 Part 4: The Data Engineering Lifecycle on AWS

4.1 Lesson Introduction

Now that you are familiar with the data engineering lifecycle and undercurrents, it is time to see how these concepts translate to tools and technologies on the AWS cloud. While other cloud providers exist—including Microsoft Azure and Google Cloud Platform, as well as smaller providers—all the concepts from this course are relevant regardless of platform. Only some tooling and implementation details differ. AWS is currently a leading cloud provider, and the course partners with them so that you can gain technical skills using the same tools and technologies that have been adopted by thousands of companies worldwide (31).

As the instructor (Morgan Willis) notes, the goal of this section is to help you “start to connect some of the concepts you’ve been learning about to the tools and technologies you’ll be practicing with in these courses.”

4.2 The Lifecycle Stages on AWS

4.2.1 Source Systems

Amazon RDS (Relational Database Service) Provisions database instances with the relational database engine of your choice, such as MySQL, PostgreSQL, MariaDB, Oracle, or SQL Server. RDS simplifies the operational overhead involved with provisioning and hosting a relational database, as well as taking care of tasks like patching and upgrading. In several labs, including the Week 2 lab, you will work with RDS as a source system (35).

Amazon DynamoDB A serverless NoSQL database option. You create standalone tables that are virtually unlimited in their total size across all items. DynamoDB has a flexible schema and is best suited for applications that require low-latency access to large volumes of data, such as gaming, IoT, mobile apps, and real-time analytics, where the data model can evolve over time without the need for complex migrations (36).

Amazon Kinesis Data Streams For streaming source systems, Kinesis enables real-time streaming of user activities—for example, from a sales platform blog. In the final week of Course 1, you will work with Kinesis Data Streams set up as a streaming source system.

Amazon SQS (Simple Queue Service) A fully managed message queuing service for handling messages between distributed application components. While not used as a source in the labs, you might use SQS in your own data pipelines (37).

Amazon MSK (Managed Streaming for Apache Kafka) Makes it easier to run Apache Kafka workloads on AWS because the underlying infrastructure is managed for you. Apache Kafka (9) is an open-source distributed event streaming platform widely used in data engineering for real-time data pipelines and streaming applications.

4.2.2 Ingestion

AWS DMS (Database Migration Service) Migrates and replicates data from a source to a target in an automated way. DMS supports both one-time migrations and ongoing replication (including CDC—Change Data Capture), making it useful when you need to continuously replicate data from a transactional database into your analytical systems (38).

AWS Glue ETL A serverless data integration service that offers features supporting data integration processes including extraction, transformation, and loading. Used in most lab exercises for ingestion. Glue supports both batch and micro-batch processing and can run Apache Spark jobs under the hood (39).

Amazon Kinesis Data Streams & Amazon Data Firehose For ingesting data from streaming sources. Kinesis Data Streams provides real-time data streaming, while Data Firehose (formerly Kinesis Data Firehose) delivers streaming data to destinations like S3, Redshift, or OpenSearch with automatic scaling and minimal configuration.

4.2.3 Storage

Amazon Redshift A cloud data warehouse service optimized for analytical queries against structured data. Redshift uses columnar storage, massively parallel processing (MPP), and result caching to deliver fast query performance at scale (41).

Amazon S3 (Simple Storage Service) Object storage that serves as the backbone of a data lake. S3 is designed for 99.999999999% (11 nines) durability and virtually unlimited scalability. Data in S3 can be organized into “folders” (prefixes) and stored in various formats including Parquet, Avro, ORC, JSON, and CSV (40).

Lakehouse Architecture Combining services such as Redshift and S3 into a **lakehouse** arrangement for seamless access to both structured data in a data warehouse and unstructured data in an object storage data lake. Technologies like Apache Iceberg (12), Delta Lake, and Apache Hudi enable ACID transactions, schema evolution, and time-travel queries on top of object storage, bridging the gap between lakes and warehouses (11).

4.2.4 Transformation

AWS Glue The primary transformation service used in these courses. Glue provides a serverless Apache Spark environment for running ETL jobs, along with a visual interface (Glue Studio) for building transformations without writing code (39).

Apache Spark A unified analytics engine for large-scale data processing (10). Can be used in combination with Glue (which runs Spark under the hood) or deployed independently on Amazon EMR (Elastic MapReduce).

dbt (data build tool) A transformation framework that enables data engineers and analysts to transform data using SQL `SELECT` statements, with built-in support for testing, documentation, and version control (13). dbt popularized the **ELT** (Extract, Load, Transform) pattern, where raw data is loaded into the warehouse first and then transformed in place.

4.2.5 Serving

Analytics Use Case

- **Amazon Athena:** An interactive query service that allows you to analyze data directly in S3 using standard SQL, with no infrastructure to manage. Athena is serverless and charges per query based on the amount of data scanned (42).
- **Amazon Redshift:** Also used for serving analytical queries, especially when data is already in the warehouse.
- **Jupyter Notebooks:** Used in the Week 2 lab for dashboards and analysis. Jupyter provides an interactive computing environment combining code, visualizations, and narrative text.
- **Amazon QuickSight:** A cloud-native, serverless BI service for creating and publishing interactive dashboards.
- **Apache Superset** and **Metabase:** Open-source dashboard and visualization tools that can connect to various data sources.

AI/Machine Learning Use Case

- Serving batch data for model training (e.g., from S3 or Redshift into Amazon SageMaker).
- Vector database options for product recommenders and use with large language models (e.g., Amazon OpenSearch, Pinecone, pgvector).

4.3 The Undercurrents on AWS

4.3.1 Security

Shared Responsibility Model As introduced in Week 1, the **shared responsibility model** says that AWS is responsible for security *of* the cloud (data centers, hardware, virtualization layer), while you are responsible for security *in* the cloud (configuring services, managing access, encrypting

data). For example, if you store data on Amazon S3, it is AWS's responsibility to make sure the S3 service itself is secure; it is your responsibility to make sure access permissions are set correctly so your data is only available to people and applications that should have access (33).

IAM (Identity and Access Management) Through IAM, you set up **roles** and **permissions** that control access to AWS resources, ensuring that users and services have the necessary—and only the necessary—access to perform their tasks securely. Within your data pipelines, you use **IAM roles**, which give users or applications access to **temporary credentials** that automatically rotate and provide appropriate AWS API permissions to various tools or data storage areas. This is a direct implementation of the Principle of Least Privilege (see Section 3.2) (34).

Network Security Services and features like **Amazon VPC** (Virtual Private Cloud) and **security groups** (instance-level firewalls) are key aspects of implementing secure data pipelines. VPCs allow you to create logically isolated private networks in the cloud, while security groups act as virtual firewalls controlling inbound and outbound traffic at the instance level.

4.3.2 Data Management

AWS Glue Crawlers & Glue Data Catalog Discover, create, and manage metadata for data stored in Amazon S3 or other storage and database systems. Glue Crawlers automatically scan data sources, identify formats and schemas, and populate the Glue Data Catalog—a centralized metadata repository that makes data discoverable and queryable by services like Athena and Redshift Spectrum (39).

AWS Lake Formation Helps centrally manage and scale fine-grained data access permissions. Lake Formation provides column-level and row-level security, data filtering, and tag-based access control, relating to both data management (data privacy and discovery) and security (43).

4.3.3 DataOps

Amazon CloudWatch Collects metrics and provides monitoring features for cloud resources, applications, and even on-premises resources. CloudWatch supports custom metrics, dashboards, and alarms that can trigger automated actions (44).

Amazon CloudWatch Logs Stores and analyzes operational logs from AWS services and custom applications. Log Insights provides a query language for searching and analyzing log data.

Amazon SNS (Simple Notification Service) Provides a means of setting up notifications—either between applications (pub/sub messaging) or via text/email—that are triggered by events within your system. SNS is commonly used in conjunction with CloudWatch Alarms to notify teams of pipeline failures or anomalies (45).

Open-Source Observability Tools Tools like Monte Carlo (data observability platform), BigEye, and Great Expectations (open-source data quality) can complement AWS-native monitoring services.

4.3.4 Orchestration

Apache Airflow / Amazon MWAA Apache Airflow (14) is the current industry-standard orchestration tool. You can implement it as an open-source tool on your own infrastructure or use **Amazon MWAA (Managed Workflows for Apache Airflow)**, a fully managed service that handles the operational overhead of running Airflow at scale.

Newer Alternatives **Dagster** (15), **Prefect**, and **Mage** aim to solve some issues that Airflow was not originally designed for, such as first-class support for data assets (rather than just task scheduling), improved local development experience, and better handling of dynamic workflows. You should be aware of these alternatives as the orchestration landscape continues to evolve.

4.3.5 Data Architecture

AWS Well-Architected Framework A set of principles and practices developed by AWS to help you build systems with an eye toward operational excellence, security, reliability, performance efficiency, cost optimization, and sustainability. The framework provides a consistent approach for evaluating architectures and implementing designs that scale over time. It is organized around six **pillars**, each with its own set of best practices and design principles (32). These pillars align closely with the data architecture principles discussed in Section 3.4.

4.3.6 Software Engineering

Amazon CodeDeploy Automates code deployment to Amazon EC2 instances, on-premises servers, Lambda functions, and ECS services, minimizing downtime during application updates.

CI/CD Tools AWS provides a suite of CI/CD services including **AWS CodePipeline** (continuous delivery), **AWS CodeBuild** (build automation), and **AWS CodeCommit** (managed Git repositories). These can be used alongside third-party tools like GitHub Actions, GitLab CI/CD, or Jenkins.

Git and GitHub Version control with Git and collaboration through GitHub remain fundamental to the software engineering practices of data engineers. All pipeline code, infrastructure definitions, and configuration should be version-controlled.

5 Week 2 Summary

This week provided a comprehensive deep dive into every stage of the data engineering lifecycle and each of the six undercurrents. The key takeaways are:

1. **Understand your source systems:** Work directly with source system owners and build good relationships to anticipate changes and avoid pipeline failures. Source systems and ingestion represent the biggest bottlenecks of the lifecycle.
2. **Choose ingestion patterns wisely:** Batch ingestion remains practical for many use cases. Adopt streaming ingestion only when a clear business use case justifies the trade-offs. In practice, batch and streaming coexist, and you choose where the boundaries occur.
3. **Be storage savvy:** Understand the full storage hierarchy—from raw ingredients (disk, SSD, RAM) through storage systems (databases, object storage) to storage abstractions (warehouses, lakes, lakehouses). Failing to understand your storage solutions can lead to costly mistakes.
4. **Add value through transformation:** Queries, modeling, and transformation are where raw data becomes useful. Understand SQL deeply, model data deliberately (normalization vs. denormalization), and recognize that transformation happens at multiple stages of the pipeline.
5. **Embrace the undercurrents:** Security (Least Privilege, defensive mindset), data management (DMBOK, data governance, data quality), DataOps (automation, observability, incident response), data architecture (nine principles, FinOps), orchestration (DAGs, Airflow), and software engineering (clean, testable code) are cross-cutting concerns that determine the quality and reliability of everything you build.
6. **Map concepts to tools:** Understanding how lifecycle stages and undercurrents map to specific AWS services—from RDS and DynamoDB for sources, through Glue and Kinesis for ingestion, S3 and Redshift for storage, to IAM and CloudWatch for security and observability—bridges the gap between theory and practice.

By internalizing this mental framework before diving into hands-on implementation, you will be better equipped to make sound architectural decisions, avoid common pitfalls, and deliver genuine value to your organization and its stakeholders.

References

- [1] J. Reis and M. Housley, *Fundamentals of Data Engineering: Plan and Build Robust Data Systems*, O'Reilly Media, 2022.
- [2] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*, O'Reilly Media, 2017.
- [3] R. Kimball and M. Ross, *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3rd ed., Wiley, 2013.
- [4] W. H. Inmon, *Building the Data Warehouse*, 4th ed., Wiley, 2005.
- [5] E. F. Codd, "A Relational Model of Data for Large Shared Data Banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.
- [6] C. J. Date, *An Introduction to Database Systems*, 8th ed., Addison-Wesley, 2003.
- [7] DAMA International, *DAMA-DMBOK: Data Management Body of Knowledge*, 2nd ed., Technics Publications, 2017.
- [8] J. Ladley, *Data Governance: How to Design, Deploy, and Sustain an Effective Data Governance Program*, 2nd ed., Academic Press, 2019.
- [9] N. Narkhede, G. Shapira, and T. Palino, *Kafka: The Definitive Guide*, O'Reilly Media, 2017.
- [10] M. Zaharia et al., "Apache Spark: A Unified Engine for Big Data Processing," *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.
- [11] M. Armbrust et al., "Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics," in *Proc. CIDR*, 2021.
- [12] Apache Software Foundation, "Apache Iceberg: An Open Table Format for Huge Analytic Datasets," 2020. [Online]. Available: <https://iceberg.apache.org/>
- [13] dbt Labs, "dbt (data build tool)," 2020. [Online]. Available: <https://www.getdbt.com/>
- [14] Apache Software Foundation, "Apache Airflow," 2015. [Online]. Available: <https://airflow.apache.org/>
- [15] Elementl, "Dagster: An Orchestration Platform for the Development, Production, and Observation of Data Assets," 2019. [Online]. Available: <https://dagster.io/>
- [16] R. T. Fielding, "Architectural Styles and the Design of Network-Based Software Architectures," Ph.D. dissertation, University of California, Irvine, 2000.
- [17] J. H. Saltzer and M. D. Schroeder, "The Protection of Information in Computer Systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.
- [18] European Parliament, "Regulation (EU) 2016/679 (General Data Protection Regulation)," *Official Journal of the European Union*, 2016.
- [19] S. Rose et al., "Zero Trust Architecture," NIST Special Publication 800-207, National Institute of Standards and Technology, 2020.
- [20] G. Kim, J. Humble, P. Debois, and J. Willis, *The DevOps Handbook*, IT Revolution Press, 2016.
- [21] K. Beck et al., "Manifesto for Agile Software Development," 2001. [Online]. Available: <https://agilemanifesto.org/>
- [22] J. P. Womack and D. T. Jones, *Lean Thinking: Banish Waste and Create Wealth in Your Corporation*, revised ed., Free Press, 2003.
- [23] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*, Addison-Wesley, 2010.

- [24] L. C. Erhard et al., “DataOps—A Systematic Literature Review,” in *Proc. Americas Conference on Information Systems (AMCIS)*, 2020.
- [25] E. Breck et al., “Data Validation for Machine Learning,” in *Proc. MLSys*, 2019.
- [26] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, *Site Reliability Engineering: How Google Runs Production Systems*, O’Reilly Media, 2016.
- [27] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*, Prentice Hall, 2008.
- [28] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, O’Reilly Media, 2015.
- [29] J. R. Storment and M. Fuller, *Cloud FinOps: Collaborative, Real-Time Cloud Financial Management*, 2nd ed., O’Reilly Media, 2023.
- [30] Y. Brikman, *Terraform: Up & Running*, 2nd ed., O’Reilly Media, 2019.
- [31] Amazon Web Services, “AWS Overview,” [Online]. Available: <https://aws.amazon.com/what-is-aws/>
- [32] Amazon Web Services, “AWS Well-Architected Framework,” [Online]. Available: <https://docs.aws.amazon.com/wellarchitected/>
- [33] Amazon Web Services, “Shared Responsibility Model,” [Online]. Available: <https://aws.amazon.com/compliance/shared-responsibility-model/>
- [34] Amazon Web Services, “AWS Identity and Access Management (IAM),” [Online]. Available: <https://aws.amazon.com/iam/>
- [35] Amazon Web Services, “Amazon Relational Database Service (RDS),” [Online]. Available: <https://aws.amazon.com/rds/>
- [36] Amazon Web Services, “Amazon DynamoDB,” [Online]. Available: <https://aws.amazon.com/dynamodb/>
- [37] Amazon Web Services, “Amazon Simple Queue Service (SQS),” [Online]. Available: <https://aws.amazon.com/sqs/>
- [38] Amazon Web Services, “AWS Database Migration Service,” [Online]. Available: <https://aws.amazon.com/dms/>
- [39] Amazon Web Services, “AWS Glue,” [Online]. Available: <https://aws.amazon.com/glue/>
- [40] Amazon Web Services, “Amazon Simple Storage Service (S3),” [Online]. Available: <https://aws.amazon.com/s3/>
- [41] Amazon Web Services, “Amazon Redshift,” [Online]. Available: <https://aws.amazon.com/redshift/>
- [42] Amazon Web Services, “Amazon Athena,” [Online]. Available: <https://aws.amazon.com/athena/>
- [43] Amazon Web Services, “AWS Lake Formation,” [Online]. Available: <https://aws.amazon.com/lake-formation/>
- [44] Amazon Web Services, “Amazon CloudWatch,” [Online]. Available: <https://aws.amazon.com/cloudwatch/>
- [45] Amazon Web Services, “Amazon Simple Notification Service (SNS),” [Online]. Available: <https://aws.amazon.com/sns/>